ROME + Coverage-Guided Testbench Refinement

LLM4ChipDesign · ROME Demo Assignment (Coverage Integration) · Anuj Apte · October 2026

1. Setup, model and settings

Notebook: `rome_submission.ipynb`, built from the original `ROME_Demo.ipynb`; its four code cells (LLM classes, `verilog_loop`, `hier_gen`, `ppa_verilog_loop`) are kept verbatim. Tools: Icarus Verilog 12.0, Covered 0.7.10, Yosys 0.70 (YoWASP build; distro Yosys 0.33 keeps the quotes of `tee -o "file"` in the file name and the original PPA script never gets its stat file). ROME settings: `prev_context_mode="interfaces"`, `max_iterations=10` (generation), 5 (RTL repair in Part III), 3 (PPA), warnings treated as failures (original behaviour).

LLM. Live mode uses the notebook default (`MODEL=Claude`, `claude-sonnet-4-6`, max_tokens 16384, API-default temperature). No API key was available on the machine used, so **the submitted outputs were produced in replay mode**: each LLM call was answered by Claude (`claude-opus-5-5`) for the exact prompt the pipeline built; prompt, SHA-256 and response are stored in `llm_responses/` and replayed by a wrapper around `generate_verilog` (30 calls). All simulation, Yosys and Covered runs are real. **One response is a labelled seeded fault**: the first `adder8` computes `ovf` for negative overflow only. With replayed answers any "first-try mistake" would be my choice anyway, so it is disclosed (README, notebook banner, JSON field) rather than presented as spontaneous. It exists to exercise Step 4 (stronger TB exposes an RTL bug).

2. Part I — original mux hierarchy and PPA

`hier_gen` produced mux2_1 → mux64_1, each level instantiating two copies of the previous mux plus a mux2_1; every module passed its supplied ROME testbench at iteration 0 (final `vvp` outputs end with `Test 5 passed!` … `All tests passed!`, logs in `part1_mux/<module>/`). PPA proxies (Yosys `stat -tech cmos`, no liberty, combinational so no timing proxy):

module	transistor est.	cells	wire bits	best iter / 3	change
mux2_1	12	1	4	0	+0.0%
mux4_1	36	3	11	0	+0.0%
mux8_1	84	7	28	0	+0.0%
mux16_1	180	15	69	0	+0.0%
mux32_1	372	31	198	0	+0.0%
mux64_1	756	63	455	0	+0.0%

Every first correct candidate already sits at the floor of this cost model (N–1 2:1 muxes, 12 transistors each); the and-or and split variants proposed in iterations 1–2 were correct but not cheaper, so the optimizer kept iteration 0. Covered on mux64_1 with the supplied 9-vector TB gives 100 % line, toggle and logic coverage (125 lines, 784 toggle bits, 345 combinations) — the all-0/all-1 vectors toggle every net, which shows these metrics say nothing about having checked all 64 select values.

3. Part II — half_adder → full_adder → adder4 → adder8

module	interface	structure	baseline TB (part2_adder/testbenches)
half_adder	a, b → sum, carry	XOR / AND	exhaustive, 4 vectors
full_adder	a, b, cin → sum, cout	2 × half_adder, cout = c1|c2	exhaustive, 8
adder4	[3:0] a, b, cin → [3:0] sum, cout	4 × full_adder ripple	exhaustive, 512
adder8 (top)	[7:0] a, b, cin → [7:0] sum, cout, ovf	2 × adder4 + ovf logic	8 directed vectors (coverage start point)

Each TB has a reference model, counts mismatches and prints `passed!` only with zero mismatches (`FAILED` otherwise). A notebook check runs all four TBs on a structural reference (all pass) and on a half_adder XOR→OR mutant (all fail: 1/4, 3/8, 390/512, 6/8). With the TBs fixed, ROME generated all four modules; each passed at iteration 0 (the seeded ovf bug is invisible to the 8-vector baseline).

Final prompt, top module (user turn after the ROME instruction prompt and the full baseline TB; previous modules given as interfaces):

```
// We are generating a 8-bit ripple-carry adder built from two adder4 instances, with carry-out cout and
// signed two's-complement overflow flag ovf hierarchically in Verilog.  // Use Verilog-2001 only.
// Previously generated modules are shown below for reference only. You may instantiate them, but ...
module half_adder(input wire a, input wire b, output wire sum, output wire carry); endmodule
module full_adder(input wire a, input wire b, input wire cin, output wire sum, output wire cout); endmodule
module adder4(input wire [3:0] a, input wire [3:0] b, input wire cin,
              output wire [3:0] sum, output wire cout); endmodule
// IMPORTANT: Output ONLY the new module requested below. ...
// Now generate: 8-bit ripple-carry adder built from two adder4 instances, with carry-out cout and ... ovf
module adder8(input wire [7:0] a, input wire [7:0] b, input wire cin, output wire [7:0] sum,
              output wire cout, output wire ovf);
// Insert code here
endmodule
```

Representative submodule (full_adder): same template with `// Now generate: full adder built from two half_adder instances (cout = carry of either half adder)`, the half_adder interface, `module full_adder(input wire a, input wire b, input wire cin, output wire sum, output wire cout);` and the exhaustive full_adder TB. Response: two half_adder instances and `assign cout = c1 | c2;`.

4. Integrating coverage_exp into ROME

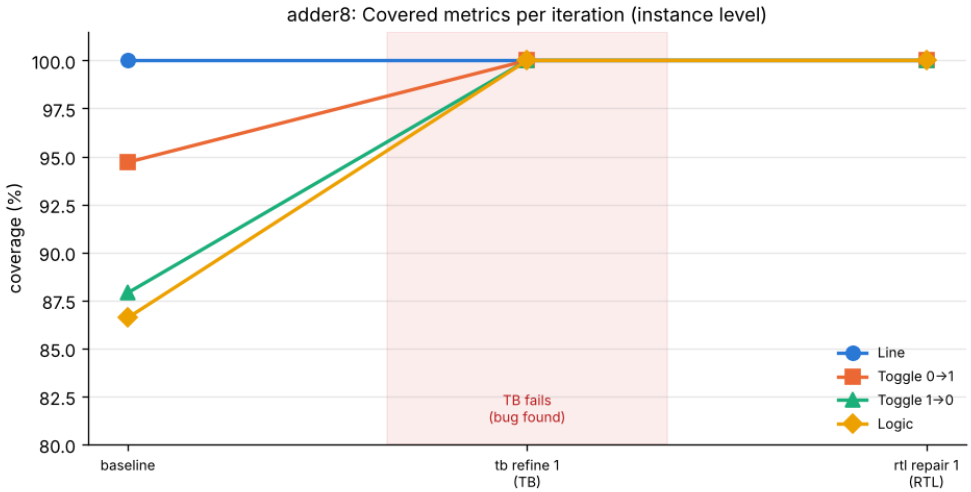
I ported the `coverage_exp` Makefile into notebook functions that any ROME hierarchy can call (`run_coverage`): (1) concatenate the hierarchy's RTL; (2) **wrap** the TB instead of editing it: a second top module `cov_dump` calls `$dumpfile/$dumpvars` and iverilog gets both roots (`-s tb_adder8 -s cov_dump`); (3) `vvp`; (4) strip the `$comment` line Covered 0.7.10 rejects, as the Makefile's perl step does; (5) `covered score -t adder8 -i tb_adder8.dut`; (6) `covered report -m ltc` summary (instance level, stricter for hierarchies) and verbose detail. `parse_detail` turns missed lines, toggle bit masks and the starred logic combinations into one-line targets (e.g. `tb_adder8.dut.hi.fa2.ha0.carry: never 0->1; never 1->0`, `lo.fa2.ha1 line 3: assign sum = a ^ b -> missing LR=01`). Each run folder keeps the scored RTL/TB, `commands.sh`, VCD, `.cdd`, reports and `coverage_feedback.txt`. On the course example it reproduces the published 60 / 64 / 21 / 35 %.

Loop separation. `CoverageLoop` has two moves: *refine_tb* (RTL frozen) sends the TB + feedback to the LLM, which may only return new `check(...)` calls; they are inserted at `// @@COVERAGE_TESTS@@` and the code asserts every old TB line survives in order. *repair_rtl* (TB frozen) calls the original `verilog_loop` with the stronger TB, the failing RTL and its vvp output, and asserts the TB hash is unchanged. Every measurement stores SHA-256 of the RTL and TB; `check_separation()` rejects any step that changed both.

5. Iterations and coverage history (adder8)

#	step	changed	TB	RTL	sim	tests	line	tog 0→1	tog 1→0	logic
0	baseline	—	tb_iter_0.v	adder8_v0.v	passed	8	100.0%	94.7%	87.9%	86.6%
1	tb_refine_1	TB only	tb_iter_1.v	adder8_v0.v	FAILED	19	100.0%	100.0%	100.0%	100.0%
2	rtl_repair_1	RTL only	tb_iter_1.v	adder8_v1.v	passed	19	100.0%	100.0%	100.0%	100.0%

Before these steps, Part II's ROME loop changed only RTL (4 generations, TBs fixed). **Baseline**: line coverage is trivially 100 % because every RTL statement is a continuous assign that executes once; the real holes were toggles and combinations: carry-generate (a&b=1) never occurred at bits 1, 2, 3, 5, 6 (so `fa*.c1` never toggled and `cout = c1 | c2` never saw c1=1), `ha1.sum` never saw "no propagate but carry-in" (LR=01) at bits 2, 3, 5, 7, and `b[6]`, `b[1]` never fell. **Refine 1** added 11 targeted vectors: a walking generate bit (02+02 … 40+40), carries arriving at bits 2/3/5/7 with a=b=0 there (03+01, 07+01, 1F+01, 7F+01), and a return-to-zero vector. Two of them, 40+40 and 7F+01, are positive signed overflows and failed: `expected ovf=1, got 0`. Following the rule, tb_iter_1.v was frozen and the ROME loop repaired adder8 (`ovf = (a7&b7&~s7) | (~a7&~b7&s7)`), passing at its first iteration. Re-measured with the same TB, all four metrics are 100 % (190/190 toggle bits each way, 153/153 combinations), so the TB loop stopped; no tb_iter_2 was needed.



6. Final commands and result

```
iverilog -g2001 -o adder8_final.vvp part3_coverage/final_rtl/{half_adder,full_adder,adder4,adder8}.v
         part3_coverage/tb_final.v
vvp adder8_final.vvp        ->  tests run: 19, mismatches: 0   passed!
# coverage (from part3_coverage/cov_02_rtl_repair_1/commands.sh)
iverilog -g2001 -Wall -s tb_adder8 -s cov_dump -o sim.vvp adder8_rtl.v tb_iter_1.v cov_dump.v ; vvp sim.vvp
```

```
covered score -t adder8 -i tb_adder8.dut -v adder8_rtl.v -vcd adder8.vcd -o adder8.cdd
covered report -m ltc -d s -i -o coverage_summary.rpt adder8.cdd
covered report -m ltc -d v -i -o coverage_detail.rpt adder8.cdd
```

The final, strongest TB (tb_final.v = tb_iter_1.v, 19 checks) passes on the final RTL with 100 % line / toggle 0→1 / toggle 1→0 / logic coverage. **Take-aways:** the Covered holes pointed straight at the untested part of the carry chain, and exercising it is what exposed the overflow bug; 100 % of these metrics is still not a proof of correctness (mux64_1 reaches it with 9 vectors covering 8 of its 64 select values), so the exhaustive submodule TBs remain important.